Performance Testing

CrimeVision — Intelligent Multimodal Forensic Analysis Platform

Date	1 November 2026
Team ID	[Enter Team ID]
Project Name	CrimeVision
Maximum Marks	5 Marks

Performance Testing

This report defines practical checks for the current CrimeVision prototype. The application runs locally in WSL 2 Ubuntu with Flask and CPU-only PyTorch. Automated tests can use pytest; pytest is suitable for correctness and API checks, but a separate load-testing tool is needed to measure concurrent-user performance.

Step 1: Testing Overview

Field	Details
Testing tool	pytest for automated functional/API checks. No load-testing tool or benchmark output has been supplied.
Type of testing	API response checks, validation checks, smoke tests, and planned lightweight load testing.
Target module / API	Case listing, case evidence listing, saved-analysis retrieval, and evidence registration endpoints.
Test environment	Local WSL 2 Ubuntu; Python virtual environment; Flask app; CPU-only PyTorch.
Test date	[Enter actual date tests are run]

Step 2: Test Scenarios

S.No	Test scenario / description	Virtual users	Duration	Expected outcome
1	Run pytest suite for route, validation, and service-level checks.	1 test process	As executed	Tests pass; failures identify the relevant endpoint or service.
2	Request the case list and a case's evidence/analysis lists repeatedly.	1 initially	60 sec (planned)	Valid responses with consistent JSON structure and no unexpected server errors.
3	Retrieve an existing saved analysis by result ID.	1 initially	60 sec (planned)	Saved analysis is returned; unknown IDs return a not-found response.
4	Upload a small valid media file and an invalid/unsupported file.	1 initially	As executed	Valid upload is registered; invalid input is rejected with an appropriate HTTP error.

Important: The durations and user counts above are proposed test settings, not measured runs. Do not report them as completed performance tests.

Step 3: Performance Test Results

No benchmark logs or pytest output were provided with the project notes. Actual values are therefore left as “Not measured” rather than estimated or invented.

S.No	Metric	Target value	Actual value	Status	Remarks
1	Average response time	< 2 seconds	Not measured	Pending	Capture from a timed run.
2	Maximum response time	< 5 seconds	Not measured	Pending	Measure across repeated requests.
3	Throughput (requests/sec)	Record baseline	Not measured	Pending	Requires timed repeated requests or a load-testing tool.
4	Error rate	< 1%	Not measured	Pending	Calculate failed requests / total requests.
5	CPU utilization	< 80%	Not measured	Pending	Monitor during a repeatable test.
6	Memory utilization	< 80%	Not measured	Pending	Monitor during a repeatable test.

Step 4: Observations & Analysis

Key findings: The project environment checks passed and a sample YOLO image-detection run was previously completed. This confirms a basic smoke test, not a performance benchmark. No response-time, throughput, or resource-utilization measurements are currently available.

Bottlenecks identified: Not yet measured. CPU-only model inference may take longer on larger media or under concurrent requests; this is a risk to test, not a confirmed bottleneck.

Optimization steps taken: The current prototype uses the installed CPU PyTorch build, so CUDA is not required. No performance-specific optimization is claimed without benchmark evidence.

Step 5: Screenshots / Evidence

Add screenshots after running the tests: (1) terminal output from **pytest -q**; (2) timed API requests or load-test report showing response times and failures. Until these are captured, mark performance results as pending.

Suggested first check: From the project root with the virtual environment active, run `pytest -q`. If pytest or the project tests are not installed/created yet, record that accurately and set up a small test suite before claiming a pass.

Submission note: Replace the team ID and test date. This report distinguishes previously verified smoke-test activity from performance metrics that still need to be measured.